

Practice 1:

Project Structure, Training, and Visualization

Course: Software Development Oriented to Machine Learning

Overview

This first practice lays the foundation for a machine learning project that will grow throughout the semester. You will organize a clean codebase, train a basic model on a public dataset, and produce visualizations that explore your data and evaluate your model's performance.

The goal is to get comfortable with ML and visualization tooling and to show good project organization. The goal is NOT to chase state-of-the-art results. Please, for your own sanity, do not focus too much on the performance of the model.

Objectives

- Structure an ML project following industry-standard conventions.
- Train a simple model on a public dataset using an established framework.
- Visually explore your dataset and communicate findings through clear charts.
- Evaluate and visualize your trained model's performance.

Requirements

1. Project Structure

Organize your code using a sensible directory structure. We recommend the Cookiecutter Data Science template at <https://cookiecutter-data-science.drivendata.org/>, though any similar professional structure that keeps data, source code, documentation, and outputs (e.g. reports/figures/) neatly separated is fine.

2. Dataset Integration

Pick a publicly available dataset from Kaggle (<https://www.kaggle.com/>) or Hugging Face Datasets (<https://huggingface.co/datasets>) that works for a supervised learning task (classification, regression, or similar). If the full dataset is bigger than 10MB, include with the code a small representative sample alongside your code for testing.

3. Framework & Training

Build your training pipeline using any established ML framework. Scikit-learn works, but we'd love to see you try PyTorch Lightning, TensorFlow, or FastAI (<https://docs.fast.ai/>).

Implement a working training loop that processes your dataset and updates model parameters. A single epoch or a few minutes of training is enough: we only want to see that the pipeline runs and that learning is happening, e.g. through logging, progress updates, or loss reports.

4. Data Exploration

Create a Jupyter notebook exploring your dataset with `matplotlib` and `seaborn`. Choose chart types appropriate to your data¹:

- **Tabular data:** histograms, scatter plots, correlation heatmaps, box plots, etc.
- **Image classification:** class distributions, image dimensions, and sample images per class (using `imshow`).

All figures must have axis labels, titles, and legends where necessary.

5. Performance Analysis

Write code (notebook or script) that evaluates your trained model and saves performance visualizations as images in your project (e.g. `reports/figures/`). Depending on your task, consider including:

- Confusion matrices (if applicable).
- Model calibration curves (see the bonus section below).
- Training losses, accuracies, etc. over time.
- Samples with the highest loss values.

6. Report

Create a minimal report (1–2 pages) presenting your key performance visualizations with brief interpretations and captions. Markdown or Typst are recommended for producing professional-looking PDFs with minimal effort, but Word or similar tools are also acceptable. Submit both the compiled PDF/HTML and the original source file.

Technical Specifications

- Package your code as a `.zip` or `.tar.gz` including source files, configuration files, a data sample, and documentation.
- Write a `README.md` explaining what the project does, what needs to be installed, the data source, and how to run the code. We should be able to follow it without guessing.
- Format figures properly (axis labels, titles, legends, captions) and save them at reasonable resolutions.
- Try to keep code modular and reusable.

Evaluation Criteria

¹Only guidelines — feel free to decide what best represents your data and project.

This practice contributes to your overall project grade but isn't graded individually, it's meant to give you feedback ahead of the final submission. After all intermediate practices are completed, a final grade will be given for the whole project. You are still required to hand in code for each submission; points will be deducted for missing ones.

Bonus: Model Calibration Analysis

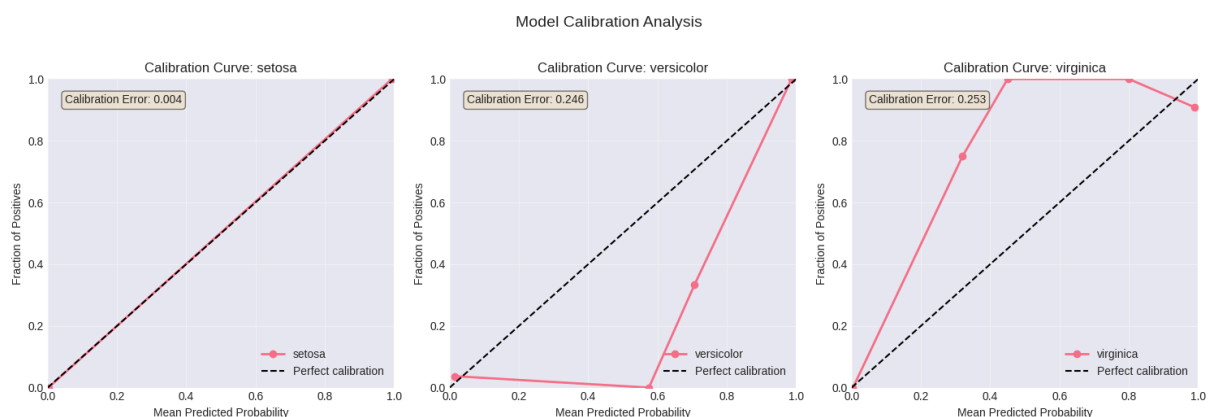
What is Model Calibration?

Model calibration measures how well a classifier's predicted probabilities match actual outcomes. A well-calibrated model is accurate in its confidence: when it predicts 80% probability for a class, roughly 80% of those predictions should actually belong to that class.

Understanding Calibration Curves

Calibration curves (reliability diagrams) bin examples by predicted probability and plot:

- X-axis: the mean predicted probability of each bin.
- Y-axis: the actual fraction of positive samples in each bin.
- Diagonal line: perfect calibration.



In the example above (a multi-class classifier on the Iris dataset), “setosa” shows near-perfect calibration, while the model is overconfident for “versicolor” and underconfident for “virginica”.

Implementation Tips

Use `scikit-learn's CalibrationDisplay.from_estimator()` to generate these plots. The calibration error quantifies deviation from perfect calibration — lower values indicate a better-calibrated model. For more on calibration methods and correction techniques, see the [scikit-learn documentation](#).